Department of Engineering Technology

Foundation University Islamabad

School of Science and Technology

Machine Learning Lab

Name: Muhammad Waleed

Roll no.: 086

Lab 04: DATA PREPROCESSING & NAIVE BAYES CLASSIFICATION

OBJECTIVES:

- To design and implement a **Spam Message Classification System** using **Naive Bayes algorithm** by converting text data into numerical form using techniques like CountVectorizer or TF-IDF.
- To evaluate the performance of the model using metrics such as **Accuracy** and **Confusion Matrix**, and test its effectiveness on new custom messages.

Performance			**Lab Report**		
Description	**Total Marks**	**Marks Obtained**	**Description**	**Total Marks**	**Marks Obtained**
Operate Software	5		Communication	5	
Code writing	5				
Total Marks obtained					

Remarks (if any): …….

Name & Signature of faculty: ……

LAB 04

Assignment Tasks:

1. **Create your own dataset with at least 20 messages (spam and ham).**

Code:

```
import pandas as pd


data = {
  "message": [
    "Win a free iPhone now",
    "Hello how are you",
    "Claim your free prize",
    "Let's meet tomorrow",
    "Exclusive offer just for you",
    "Call me when you are free",
    "You won lottery",
    "Important meeting at 5pm",
    "Free gift card available",
    "Are you coming today",
    "Congratulations you won cash",
    "Let's study together",
    "Click here to win money",
    "Dinner tonight?",
    "Limited time offer",
    "Project submission deadline",
    "Earn money fast",
    "See you in class",
    "Free entry in contest",
    "Good morning"
  ],
```

```
  "label": [
    "spam","ham","spam","ham","spam","ham","spam","ham","spam","ham",
    "spam","ham","spam","ham","spam","ham","spam","ham","spam","ham"
  ]
}

df = pd.DataFrame(data)
print(df.head())
```

Output:

```
                        message label
0         Win a free iPhone now  spam
1             Hello how are you   ham
2         Claim your free prize  spam
3           Let's meet tomorrow   ham
4  Exclusive offer just for you  spam
```

2. Perform data cleaning:

o Convert text to lowercase

o Remove special characters

Code:

```
import re

def clean_text(text):
  text = text.lower()                  # lowercase
  text = re.sub(r'[^a-z\s]', '', text)     # remove special characters
  return text

df["cleaned"] = df["message"].apply(clean_text)

print(df[["message", "cleaned"]])
```

Output:

	message	cleaned
0	Win a free iPhone now	win a free iphone now
1	Hello how are you	hello how are you
2	Claim your free prize	claim your free prize
3	Let's meet tomorrow	lets meet tomorrow
4	Exclusive offer just for you	exclusive offer just for you
5	Call me when you are free	call me when you are free
6	You won lottery	you won lottery
7	Important meeting at 5pm	important meeting at pm
8	Free gift card available	free gift card available
9	Are you coming today	are you coming today
10	Congratulations you won cash	congratulations you won cash
11	Let's study together	lets study together
12	Click here to win money	click here to win money
13	Dinner tonight?	dinner tonight
14	Limited time offer	limited time offer
15	Project submission deadline	project submission deadline
16	Earn money fast	earn money fast
17	See you in class	see you in class
18	Free entry in contest	free entry in contest
19	Good morning	good morning

3. Convert text into numerical format using CountVectorizer or TF-IDF.

Code:

```
from sklearn.feature_extraction.text import CountVectorizer


vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df["cleaned"])


y = df["label"]
```

4. Train a Naive Bayes model.

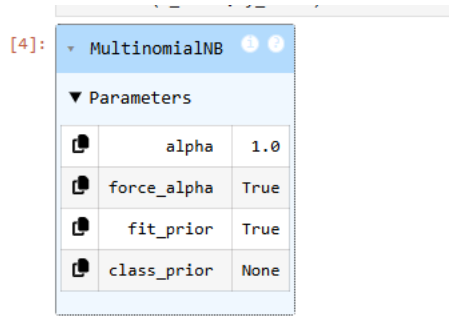
Code:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# Train model
model = MultinomialNB()
model.fit(X_train, y_train)
```

Output:



5. Evaluate your model using:

o Accuracy

o Confusion Matrix

Code:

```
from sklearn.metrics import accuracy_score, confusion_matrix


# Predictions
y_pred = model.predict(X_test)


# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)


# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

Output:

```
Accuracy: 0.6666666666666666
Confusion Matrix:
 [[2 2]
 [0 2]]
```

6. Test your model with at least 3 custom messages and show predictions.

Code:

```
test_messages = [
   "Win money now",
```

```
    "Let's go for dinner",
    "Free prize available"
]

# Clean test messages
cleaned_test = [clean_text(msg) for msg in test_messages]

# Convert to vector
test_vector = vectorizer.transform(cleaned_test)

# Predict
predictions = model.predict(test_vector)

# Show results
for msg, pred in zip(test_messages, predictions):
    print(f"Message: {msg} --> Prediction: {pred}")
```

Output:

```
Message: Win money now --> Prediction: spam
Message: Let's go for dinner --> Prediction: ham
Message: Free prize available --> Prediction: spam
```

THE END